

React Hooks

1.UseState

Before Hooks:

- Only class components could have state.
- Functional components were just UI functions.

After Hooks:

- Functional components can now:
 - Hold state
 - Handle lifecycle
 - Manage side effects
 - Replace most class components

```
const [state, setState] = useState(initialValue);
```

What happens here?

- state → current value
- setState → function to update it
- initialValue → used only during first render

```
import { useState } from "react";
```

```
function Counter() {  
  const [count, setCount] = useState(0);  
  
  return (  
    <div>  
      <h1>{count}</h1>  
      <button onClick={() => setCount(count + 1)}>  
        Increment  
      </button>  
    </div>  
  )  
}
```

```
);  
}
```

What happens internally?

1. First render → count = 0
2. Button click → setCount(1)
3. React schedules re-render
4. Component runs again
5. count = 1

When setState is called:

- React does NOT immediately update
- It schedules a re-render
- On next render, state is updated

5 Functional Updates (Very Important)

Bad way:

```
setCount(count + 1);
```

Correct way when based on previous state:

```
setCount(prev => prev + 1);
```

Why?

Because React may batch updates.

Example:

```
setCount(count + 1);  
setCount(count + 1);
```

Result → only +1 (not +2)

Correct:

```
setCount(prev => prev + 1);  
setCount(prev => prev + 1);
```

Result → +2

This is crucial in real-world apps.

⚠ Never mutate directly:

```
items.push(newItem); ❌
```

React needs new reference.

❌ Direct mutation

```
state.name = "New";  
setState(state);
```

React won't re-render properly.

```
setCount(5);  
console.log(count); // still old value
```

Because state updates are async.

Lazy Initialization (Performance Optimization)

If initial state is expensive:

```
const [value, setValue] = useState(() => {  
  return heavyCalculation();  
});
```

The function runs only once (first render).

Q. Why doesn't useState merge objects?

Merging is easy to do manually with spread — but impossible to “un-merge” if React did it automatically.

Q. Why must hooks be top-level?

React tracks hooks by call order in an hooks array, not by name.

If hooks were inside conditions or loops:

- The order could change between renders
- React would assign the wrong state to the wrong hook
- State corruption would happen

So hooks must run:

- In the same order
- On every render
- At top level

Q. Why do we use functional updates?

Because state updates are:

- Asynchronous
- Batched

Using:

```
setCount(prev => prev + 1);
```

Guarantees you're working with the latest state, even if multiple updates are queued.

Without it, you risk stale closures.

Q. What is lazy initialization?

Instead of:

```
useState(expensiveFunction());
```

You use:

```
useState(() => expensiveFunction());
```

React only calls the function on first render, not every render.

It improves performance for expensive computations.

Q. How does React preserve state between renders?

React stores state in an internal hook list attached to the component's fiber node.

Each render:

- **React walks through hooks in order**
- **Matches them by position**
- **Returns stored state**

State isn't stored in the function — it's stored in React's internal data structure.

Q. Why does React rely on closure for state?

Each render creates a new function scope.

Event handlers capture the state value from that render via closure.

Q. Why is state immutable?

Because React detects changes using reference comparison.

If you mutate:

state.name = "New"

The reference doesn't change → React may not re-render.

useEffect

First: What Is a “Side Effect”?

A side effect is anything that:

- Talks to the outside world
- Changes something outside render
- Runs after UI calculation

Examples:

- Fetching data
- Subscribing to events
- Setting timers
- Updating DOM manually
- Logging

useEffect is a way to tell React:

“After you finish rendering and committing changes to the DOM, run this side-effect logic.”

It is not a lifecycle replacement.

It is a synchronization primitive.

Why useEffect Exists

React components must stay pure during render.

That means:

```
function Component() {  
  // ✗ Don't fetch data here  
  // ✗ Don't start timers here  
  // ✗ Don't manipulate DOM here  
  
  return <div>Hello</div>  
}
```

Rendering should only calculate UI.

So React gives us:

```
useEffect(() => {  
  // Side effects go here  
})
```

Effects run after the render is committed to the DOM.

Basic Syntax

```
useEffect(() => {  
  // effect logic  
  
  return () => {  
    // cleanup logic (optional)  
  }  
}, [dependencies])
```

It has 3 parts:

1. Effect function
2. Cleanup function (optional)
3. Dependency array

Case 1: No Dependency Array

```
useEffect(() => {  
  console.log("Runs after every render")  
})
```

👉 Runs after every render

Why?

Because React doesn't know when to skip it.

Case 2: Empty Dependency Array []

```
useEffect(() => {  
  console.log("Runs once on mount")  
}, [])
```

👉 Runs only once (on mount)

Use this for:

- API calls
- Initial setup
- Subscriptions

Case 3: With Dependencies

```
useEffect(() => {  
  console.log("Runs when count changes")  
}, [count])
```

👉 Runs:

- On mount
- Whenever count changes

Cleanup Function

```
useEffect(() => {  
  const interval = setInterval(() => {  
    console.log("Tick")  
  }, 1000)  
  
  return () => {  
    clearInterval(interval)  
  }  
}, [])
```


Cleanup runs:

- Before the effect re-runs
- When component unmounts

This prevents:

- Memory leaks
- Duplicate timers
- Multiple subscriptions

Think of it like this:

1. Render happens
2. DOM updates
3. React runs effects
4. If dependencies change → cleanup → run effect again

Flow:

Render → Commit → Effect

If dependency changes:

Render → Commit → Cleanup → Effect

Common Mistake: Missing Dependencies

```
useEffect(() => {  
  console.log(count)  
}, []) // ❌ count missing
```

This creates a stale closure bug.

The effect captures the old value of count.

React's rule:

If you use it inside the effect, it must be in the dependency array.

Q.What is useEffect and why do we use it?

Answer (Interview Ready):

useEffect is a React Hook used to perform side effects in functional components. Side effects include data fetching, subscriptions, timers, logging, or DOM manipulation.

It runs after the component renders and commits to the DOM.

It replaces lifecycle methods like:

- **componentDidMount**
- **componentDidUpdate**
- **componentWillUnmount**

But conceptually, it's better understood as a synchronization mechanism, not a lifecycle replacement.

Q. When does useEffect run?

Answer:

It depends on the dependency array:

- **No dependency array → runs after every render.**
- **Empty array [] → runs once on mount.**
- **[dep] → runs on mount and whenever dep changes.**

Q. What is the dependency array and why is it important?

Answer:

The dependency array tells React when to re-run the effect.

React tracks values listed in the array. If any value changes between renders, the effect runs again.

If a variable is used inside the effect but not listed in dependencies, it may cause a stale closure bug.

Rule:

Every reactive value used inside the effect must appear in the dependency array.

Q. What is the difference between `useEffect` and `useLayoutEffect`?

Answer:

Both run after render, but timing differs.

- `useEffect` runs after the browser paints
- `useLayoutEffect` runs before the browser paints

`useLayoutEffect` blocks painting.

Q. Can we use `async` directly inside `useEffect`?

Answer:

No. The effect callback cannot be `async` because it must return either:

- Nothing
- A cleanup function

This is wrong:

```
useEffect(async () => {}) ❌
```

Correct:

```
useEffect(() => {  
  async function fetchData() {  
    const res = await fetch(...)  
  }  
  fetchData()  
}, [])
```

Q. Is useEffect synchronous or asynchronous?

Answer:

useEffect runs asynchronously after the DOM commit phase.

It does not block rendering.

React prioritizes UI rendering first.

Q. What is the real purpose of useEffect in React architecture?

Interview-Ready Answer:

The real purpose of useEffect is to synchronize React state with external systems.

React components should remain pure during render. useEffect allows side effects to be executed after commit without violating purity.

It's part of React's declarative synchronization model.

UseRef

useRef is one of the most misunderstood hooks in React

useRef is a persistent mutable container that survives renders without causing re-renders.

Most devs think:

“It’s for accessing DOM.”

That’s only 20% of the story.

1 What useRef Actually Is

```
const ref = useRef(initialValue)
```

It returns:

```
{
  current: initialValue
}
```

That object:

-  Persists across re-renders
-  Is mutable
-  Does NOT trigger re-renders when changed

That last point is critical.

Why It Exists

In React, every render:

- Re-runs your component function
- Recreates local variables
- Recalculates JSX

So this:

```
let count = 0
```

Resets every render.

But useRef:

```
const countRef = useRef(0)
```

React stores it in the fiber internally.

It survives renders.

Think of it as:

“Instance variable for function components.”

Like this.someVar in class components.

Classic Use Case — DOM Access

Example: Focus Input

```
import { useRef } from "react"
```

```
function Example() {
```

```
  const inputRef = useRef(null)
```

```
  const focusInput = () => {
```

```
    inputRef.current.focus()
```

```
  }
```

```
  return (
```

```
    <>
```

```
    <input ref={inputRef} />
```

```
    <button onClick={focusInput}>Focus</button>
```

```
  </>
```

```
  )
```

```
}
```

What Happens?

- React assigns DOM node to inputRef.current
- You access it imperatively

Advanced Use Case — Storing Mutable Values Without Re-render

Example: Render Counter

```
const renderCount = useRef(0)
```

```
renderCount.current++
```

Even if it increments, UI does not re-render.

Why?

Because React does not track ref changes.

Real Power — Avoiding Stale Closures

This is where mid-level devs struggle.

The Problem

```
useEffect(() => {  
  const id = setInterval(() => {  
    console.log(count)  
  }, 1000)  
}, [])
```

count becomes stale.

Solution Using Ref

```
const countRef = useRef(count)
```

```
useEffect(() => {  
  countRef.current = count  
}, [count])
```

```
useEffect(() => {  
  const id = setInterval(() => {
```

```
console.log(countRef.current)  
, 1000)
```

```
return () => clearInterval(id)  
, [])
```

Now interval always sees latest value.

When NOT To Use useRef

- ✗ To store UI data that should update screen
- ✗ As a replacement for state
- ✗ For derived values
- ✗ To bypass React lifecycle logic

If UI must update → useState.

Ref Does NOT Cause Re-render — Why?

React re-renders when:

- State changes
- Props change
- Context changes

React does NOT track:

ref.current = something

Because ref is just a mutable object.

React doesn't proxy or observe it.

There are 3 categories of data in React:

1. Render Data → affects UI → useState
2. Persistent Instance Data → does not affect UI → useRef
3. Derived Data → computed → useMemo

UseMemo

useMemo is a React Hook that memoizes the result of a computation and recomputes it only when its dependencies change.

It is primarily used to:

- Avoid expensive recalculations
- Maintain stable references (objects/arrays) across renders

It memoizes a value, not a function.

What it does

useMemo memoizes the result of a computation.

```
const memoizedValue = useMemo(() => {  
  return expensiveComputation(a, b);  
}, [a, b]);
```

It:

- Runs the function
- Caches the result
- Recomputes only when dependencies change

◆ Why React Needs This

React re-renders a component:

- When state changes
- When props change
- When parent re-renders

On every render, everything inside the component runs again.

That includes:

- Expensive calculations

- Object creations
- Array filtering
- Sorting
- Derived data

Without `useMemo`, those run every render.

Example Without `useMemo`

```
const filteredUsers = users.filter(user => user.isActive);
```

This runs every render.

If:

- users has 10,000 items
- Component re-renders frequently

You're wasting CPU.

◆ Same With `useMemo`

```
const filteredUsers = useMemo(() => {  
  return users.filter(user => user.isActive);  
}, [users]);
```

It recalculates only when users changes.

What Actually Gets Memoized?

Important:

`useMemo` does NOT memoize the function.

It memoizes the return value.

. Stable reference for child components

This is subtle.

```
const obj = { name: "Prajwal" };
```

This creates a NEW object every render.

Even if content is same, reference is different.

If passed to a child wrapped in React.memo, child re-renders.

```
const obj = useMemo(() => ({ name: "Prajwal" }), []);
```

Now reference stays stable.

useCallback

useCallback memoizes a function reference and returns the same function instance unless its dependencies change.

It is mainly used to:

- Prevent unnecessary re-renders of memoized child components
- Maintain stable function references

It memoizes a function, not its result.

What it does

useCallback memoizes a function reference.

```
const memoizedFn = useCallback(() => {  
  doSomething(a);  
}, [a]);
```

It returns the SAME function instance unless dependencies change.

Functions are objects in JavaScript.

Every render:

```
const handleClick = () => {  
  console.log("Clicked");  
};
```

This is a NEW function every time.

Even if body is identical.

Problem Scenario

```
const Child = React.memo(({ onClick }) => {  
  console.log("Child rendered");  
  return <button onClick={onClick}>Click</button>;  
});
```

Parent:

```
const handleClick = () => {  
  console.log("Clicked");  
};
```

```
<Child onClick={handleClick} />
```

Child re-renders every time parent re-renders.

Why?

- ❓ handleClick is new reference each render.
- ❓ React.memo does shallow comparison.
- ❓ Reference changed → re-render.

Fix With useCallback

```
const handleClick = useCallback(() => {  
  console.log("Clicked");  
}, []);
```

Now:

- Same function reference.
- Child won't re-render unnecessarily.

❌ “useMemo prevents re-render”

No.

It only memoizes a value.

It does NOT stop component re-render.

❌ “Always wrap everything in useCallback”

No.

It adds overhead.

Memoization itself costs CPU and memory.

Dependency Array Deep Dive

```
useMemo(() => compute(a, b), [a, b]);
```

React:

- Stores previous deps
- Does shallow comparison
- If any dep changed → recompute

Important:

- Objects and arrays compare by reference
- Not by deep equality

If interviewer asks:

What is the difference between `useMemo` and `useCallback`?

Answer:

- `useMemo` memoizes a computed value.
- `useCallback` memoizes a function reference.
- Both rely on dependency arrays.
- They optimize referential equality to prevent unnecessary recalculations or re-renders.
- Overuse can hurt performance.

Q. What problem does `useCallback` solve with `React.memo`?

Answer:

`React.memo` does shallow comparison of props.

Since functions are objects, a new function reference is created on every render.

Without `useCallback`, the child re-renders because:

- The function reference changes.

With useCallback, the reference stays stable, so the child avoids unnecessary re-rendering.

Q. Is it good practice to wrap every function in useCallback?

Answer:

No.

Overusing useCallback:

- **Adds memory overhead**
- **Increases complexity**
- **May hurt performance**

Memoization itself has a cost.

Only use it when:

- **There is a measurable performance issue**
- **The function is passed to memoized children**

Q. What's the difference between React.memo, useMemo, and useCallback?

You say:

- **React.memo memoizes a component to prevent unnecessary re-renders when props don't change.**
- **useMemo memoizes a computed value inside a component.**
- **useCallback memoizes a function reference inside a component.**
- **All rely on shallow comparison and referential equality.**
- **They are performance optimization tools, not default coding patterns.**

useReducer

useReducer is a React hook that lets you manage complex state logic in a predictable way.

It's an alternative to useState when:

- The state is complex (nested objects, arrays, etc.).
- The next state depends on the previous state.
- You want a centralized and predictable way to handle state updates.
- You want to avoid prop drilling in larger applications when combined with

Basic Syntax

```
const [state, dispatch] = useReducer(reducer, initialState, init);
```

- state → the current state value.
- dispatch → a function used to send an action to the reducer.
- reducer → a function that takes (state, action) and returns a new state.
- initialState → the initial value of your state.
- init → optional function for lazy initialization.

```
const initialState = { count: 0 };
```

```
function reducer(state, action) {  
  switch (action.type) {  
    case 'increment':  
      return { count: state.count + 1 };  
    case 'decrement':  
      return { count: state.count - 1 };  
    case 'reset':  
      return { count: 0 };  
  }  
}
```



```
default:  
return state;  
}  
}
```

```
const [state, dispatch] = useReducer(reducer, initialState);
```

```
return (  
  <div>  
    <p>Count: {state.count}</p>  
    <button onClick={() => dispatch({ type: 'increment' })}>+</button>  
    <button onClick={() => dispatch({ type: 'decrement' })}>-</button>  
    <button onClick={() => dispatch({ type: 'reset' })}>Reset</button>  
  </div>
```